



INE-PYTHON

LECTURE - 08 FILES AND EXCEPTIONS

What You Will Learn

- File Input and Output
- Using Loop to Process Files
- Processing Records
- Working with Binary File and Record

FILES



Introduction

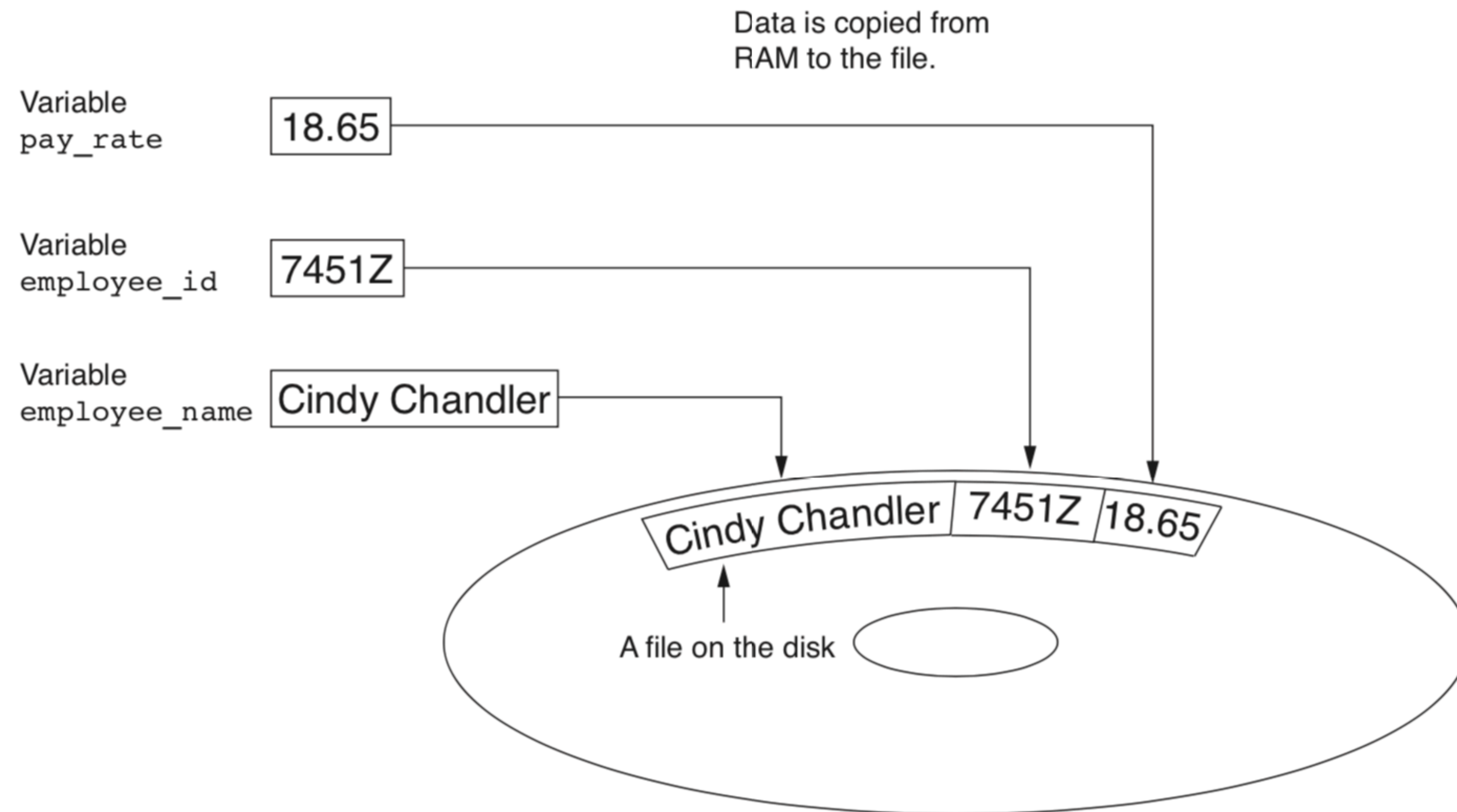
FILE INPUT/OUTPUT

สำหรับ โปรแกรมที่จะเก็บข้อมูลระหว่างช่วงเวลาที่โปรแกรมทำงาน หากต้องการให้ข้อมูลเก็บไว้ใช้ต่อเมื่อเริ่มใช้งาน

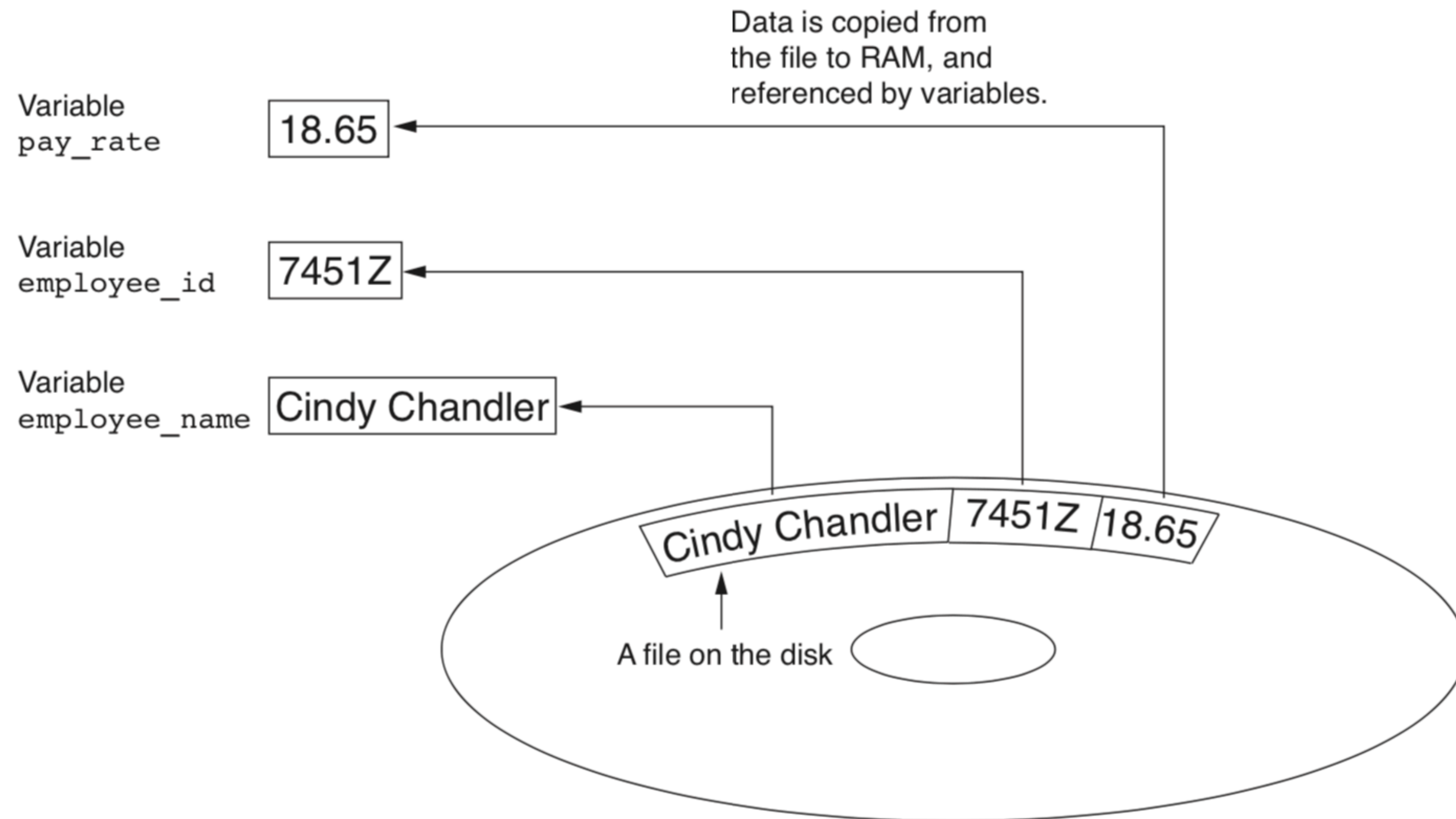
โปรแกรมครั้งใหม่ จำเป็นต้องบันทึกข้อมูล

- ข้อมูลจะถูกบันทึกลงในไฟล์ ซึ่งมักจะอยู่บนดิสก์คอมพิวเตอร์
- ข้อมูลที่บันทึกไว้สามารถเรียกคืนและนำมาใช้ในภายหลังได้
- "การเขียนข้อมูลไปยัง" หมายถึง การบันทึกข้อมูลลงในไฟล์
- ไฟล์เอาต์พุต: ไฟล์ที่มีการเขียนข้อมูลลงไป
- "การอ่านข้อมูลจาก" หมายถึง กระบวนการเรียกข้อมูลจากไฟล์
- ไฟล์อินพุต: ไฟล์ที่ใช้ในการอ่านข้อมูลออกมา

WRITE DATA TO FILE



READ DATA TO FILE



TYPE OF FILES

โดยทั่วไปมีไฟล์อยู่สองประเภท

- ไฟล์ข้อความ (Text file): ประกอบด้วยข้อมูลที่ถูกเข้ารหัสเป็นข้อความ
- ไฟล์ไบนารี (Binary file): ประกอบด้วยข้อมูลที่ยังไม่ได้แปลงเป็นข้อความ

มีสองวิธีในการเข้าถึงข้อมูลที่เก็บในไฟล์

- การเข้าถึงแบบลำดับ (Sequential access): อ่านไฟล์ทีละลำดับตั้งแต่ต้นจนจบ ไม่สามารถข้ามไปข้างหน้าได้
- การเข้าถึงแบบตรง (Direct access): สามารถข้ามไปยังข้อมูลส่วนใดก็ได้ในไฟล์โดยตรง

FILE

มีขั้นตอนสามขั้นตอนที่ต้องทำเสมอเมื่อ โปรแกรม ใช้ไฟล์

- **เปิดไฟล์ (Open the file)** - การเปิดไฟล์จะสร้างการเชื่อมต่อระหว่างไฟล์กับ โปรแกรม การเปิดไฟล์สำหรับเขียนข้อมูล (output file) มักจะสร้างไฟล์บนดิสก์และอนุญาตให้โปรแกรมเขียนข้อมูลลงไปได้ การเปิดไฟล์สำหรับอ่านข้อมูล (input file) จะทำให้โปรแกรมสามารถอ่านข้อมูลจากไฟล์ได้
- **ประมวลผลไฟล์ (Process the file)** - ในขั้นตอนนี้จะเป็นการเขียนข้อมูลลงในไฟล์ (ถ้าเป็นไฟล์สำหรับเขียน) หรืออ่านข้อมูลจากไฟล์ (ถ้าเป็นไฟล์สำหรับอ่าน)
- **ปิดไฟล์ (Close the file)** - เมื่อโปรแกรมใช้งานไฟล์เสร็จแล้ว ต้องปิดไฟล์เพื่อยกเลิกการเชื่อมต่อระหว่างไฟล์กับ โปรแกรม

KEY CONCEPTS OF FILES

แนวคิดสำคัญเกี่ยวกับไฟล์:

- การเปิดไฟล์: `open(filename, mode)`
- โหมดของไฟล์:
 - 'r': อ่าน
 - 'w': เขียน
 - 'a': เพิ่มข้อมูล
 - 'b': โหมดไบนารี
 - 't': โหมดข้อความ
 - '+': อ่าน/เขียน
- การปิดไฟล์: `file.close()`
- การใช้ with statement: ปิดไฟล์ให้อัตโนมัติ
- การอ่านไฟล์:
 - `read()`: อ่านทั้งไฟล์
 - `readline()`: อ่านทีละบรรทัด
 - `readlines()`: อ่านทุกบรรทัด
- การเขียนไฟล์: `file.write()`
- การเพิ่มข้อมูล: ใช้โหมด 'a'

KEY CONCEPTS OF FILES

```
1 file_object = open(file_name, mode)
```

โหมดต่าง ๆ:

- 'r': โหมดอ่าน (ค่าเริ่มต้น) เปิดไฟล์เพื่ออ่าน
- 'w': โหมดเขียน เปิดไฟล์เพื่อเขียน (สร้างไฟล์ใหม่หรือล้างข้อมูลไฟล์เดิม)
- 'a': โหมดเพิ่มข้อมูล เปิดไฟล์เพื่อเพิ่มข้อมูล (สร้างไฟล์ใหม่หากยังไม่มี)
- 'b': โหมดไบนารี เปิดไฟล์ในรูปแบบไบนารี
- 't': โหมดข้อความ (ค่าเริ่มต้น) เปิดไฟล์ในรูปแบบข้อความ
- '+': โหมดอัปเดต เปิดไฟล์เพื่ออ่านและเขียน

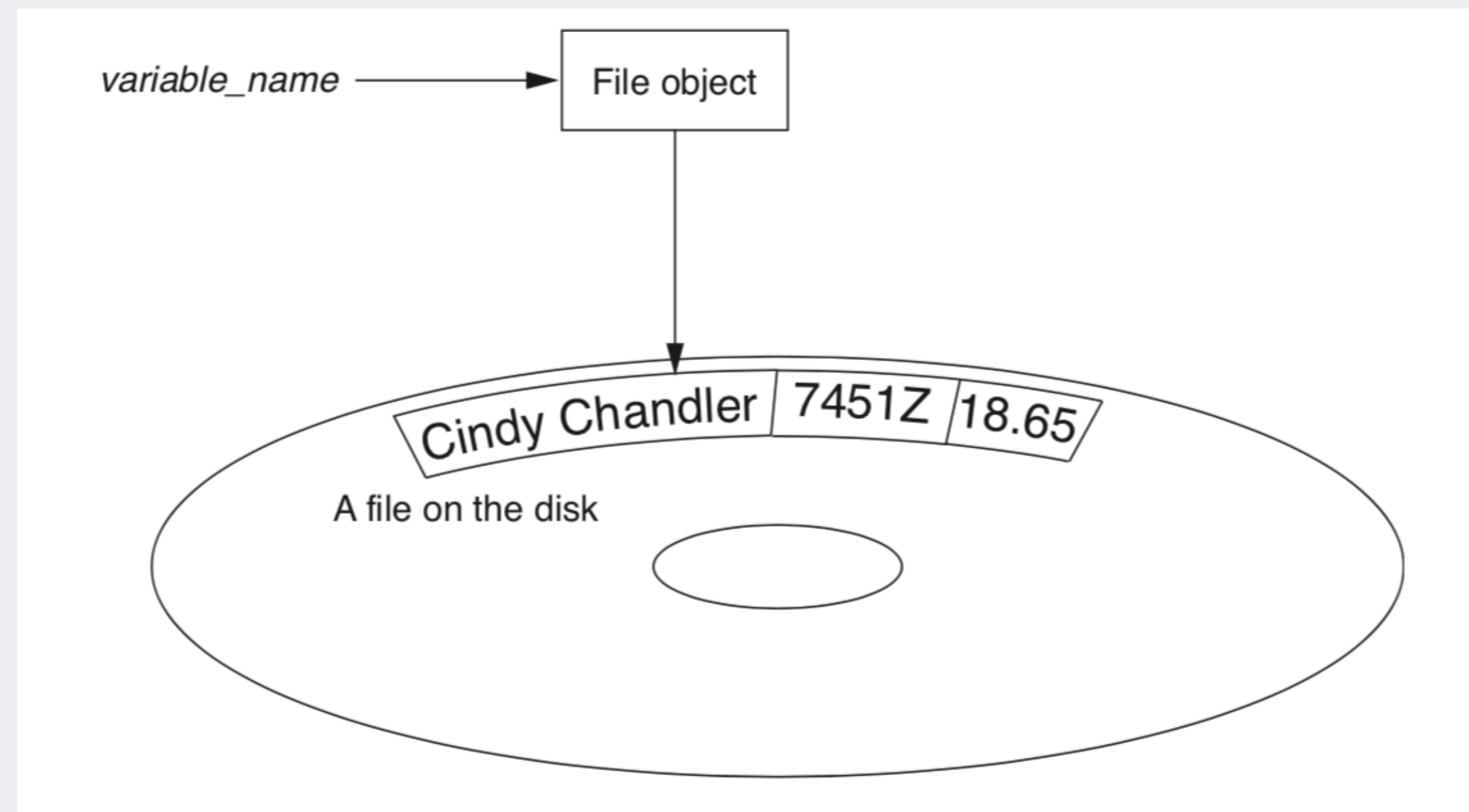
FILES



Basic Input / Output

OPEN FILE

```
file_variable = open(filename , mode)
```



OPEN AND CLOSE FILES

```
1 # Opening a file in read mode
2 file = open("example.txt", "r")
```

Listing 8.1: Example of Opening File

```
1 # Closing the file
2 file.close()
```

Listing 8.2: Example of Closing File

FILEWRITE.PY

```
1  # This program writes three lines of data
2  # to a file.
3  def main():
4      # Open a file named philosophers.txt.
5      outfile = open('philosophers.txt', 'w')
6
7      # Write the names of three philosophers
8      # to the file.
9      outfile.write('John Locke\n')
10     outfile.write('David Hume\n')
11     outfile.write('Edmund Burke\n')
12
13     # Close the file.
14     outfile.close()
15
16 # Call the main function.
17 main()
```

Open file →

Write file →

Close file →

philosophers.txt

```
1 John Locke
2 David Hume
3 Edmund Burke
```

FILEREAD.PY

```
1  # This program reads and displays the contents
2  # of the philosophers.txt file.
3  def main():
4      # Open a file named philosophers.txt.
5      infile = open('philosophers.txt', 'r')
6
7      # Read the file's contents.
8      file_contents = infile.read()
9
10     # Close the file.
11     infile.close()
12
13     # Print the data that was read into
14     # memory.
15     print(file_contents)
16
17     # Call the main function.
18     main()
```

Open file →

Read whole file →

Close file →

FILES



Using with Statement

WITH STATEMENT

The **with** statement is used to ensure that a file is properly closed after its suite finishes, even if an exception is raised. By using the **with** statement, you can manage file resources more efficiently, as it automatically handles closing the file when the block of code is exited. This eliminates the need for an explicit **close()** call, reducing the risk of leaving files open and leading to cleaner, more readable code.

```
1 # Using with statement to open and close a file
2 with open("example.txt", "r") as file:
3     contents = file.read()
4     print(contents)
5 # No need to explicitly close the file
```

Listing 8.3: Using with statement to open and close a file

WRITING TO A FILE

```
# Writing to a file
with open("example.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("This is a new line.\n")
```

Listing 8.7: Writing to a File

APPENDING TO A FILE

```
# Appending to a file
with open("example.txt", "a") as file:
    file.write("This line is appended.\n")
```

Listing 8.8: Appending to a File

READING FILE CONTENT

```
# Using with statement to open and close a file
with open("example.txt", "r") as file:
    contents = file.read()
    print(contents)
# No need to explicitly close the file
```

Listing 8.3: Using with statement to open and close a file

FILES



Using Loop to Process File

DETECTING END OF FILE

Open the file

Use readline to read the first line from the file

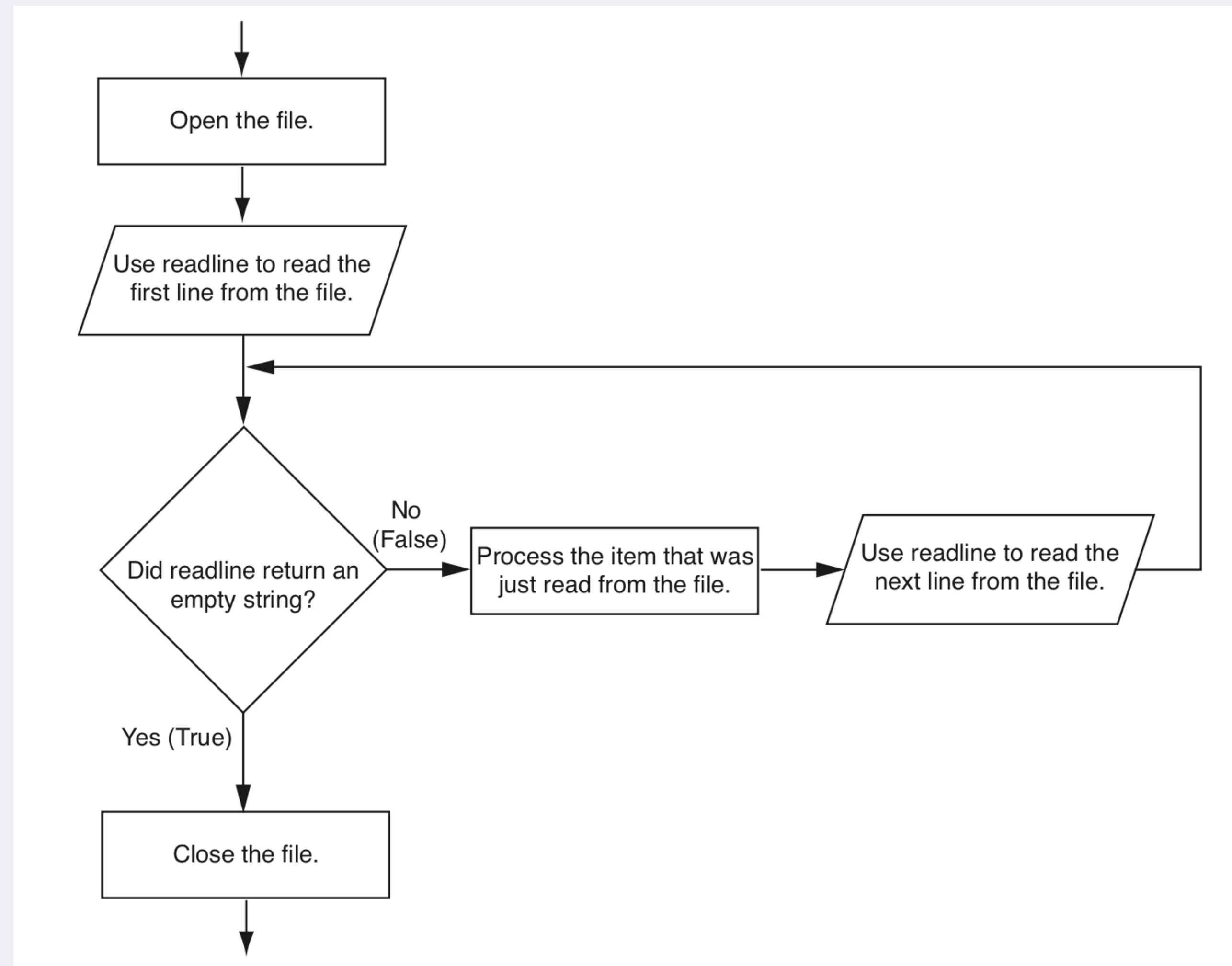
While the value returned from readline is not an empty string:

Process the item that was just read from the file

Use readline to read the next line from the file.

Close the file

DETECTING END OF FILE



READING ONE LINE AT A TIME

```
with open("example.txt", "r") as file:  
    line = file.readline()  
    while line:  
        print(line.strip())  
        line = file.readline()
```

Listing 8.5: Reading one line at a time

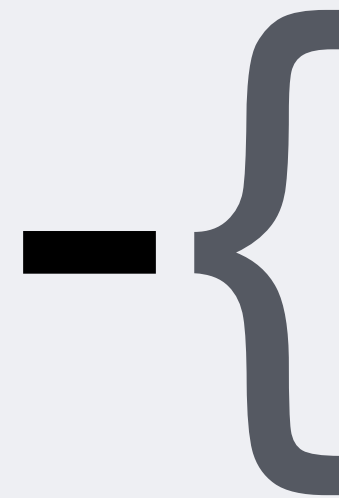
WITH STATEMENT

```
with open("example.txt", "r") as file:  
    lines = file.readlines()  
    for line in lines:  
        print(line.strip())
```

Listing 8.6: Reading all lines into a list

WRITE_SALES.PY

Write file loop



```
1 # This program prompts the user for sales amounts
2 # and writes those amounts to the sales.txt file.
3 # Get the number of days.
4 num_days = int(input('For how many days do you have sales? '))
5 # Open a new file named sales.txt using with statement.
6 with open('sales.txt', 'w') as sales_file:
7     # Get the amount of sales for each day and write it to the file.
8     for count in range(1, num_days + 1):
9         # Get the sales for a day.
10        sales = float(input(f'Enter the sales for day #{count}: '))
11        # Write the sales amount to the file.
12        sales_file.write(str(sales) + '\n')
13
14 print('Data written to sales.txt.')
```

Listing 8.9: Loop Writing File

READ_SALES2.PY(FOR LOOP)

```
1 # This program uses the for loop to read
2 # all of the values in the sales.txt file.
3 # Open the sales.txt file for reading using with statement.
4 with open('sales.txt', 'r') as sales_file:
5     # Read all the lines from the file.
6     for line in sales_file:
7         # Convert line to a float.
8         amount = float(line)
9         # Format and display the amount.
10        print(format(amount, '.2f'))
```

Read file loop — {

Listing 8.10: For Loop Reading File

READ_SALES.PY (WHILE LOOP)

Read file loop — {

```
1 # This program reads all of the values in the sales.txt file.
2 # Open the sales.txt file for reading using with statement.
3 with open('sales.txt', 'r') as sales_file:
4     # Read the first line from the file, but don't convert to a number yet.
5     # We still need to test for an empty string.
6     line = sales_file.readline()
7     # Read until the empty string is return.
8     while line != '':
9         # Convert line to a float.
10        amount = float(line)
11        # Format and display the amount.
12        print(format(amount, '.2f'))
13        # Read the next line.
14        line = sales_file.readline()
```

Listing 8.11: Using While Loop Reading File

FILES

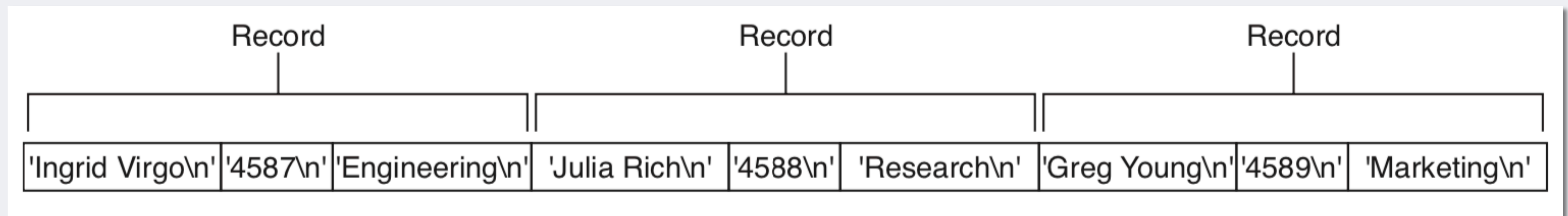
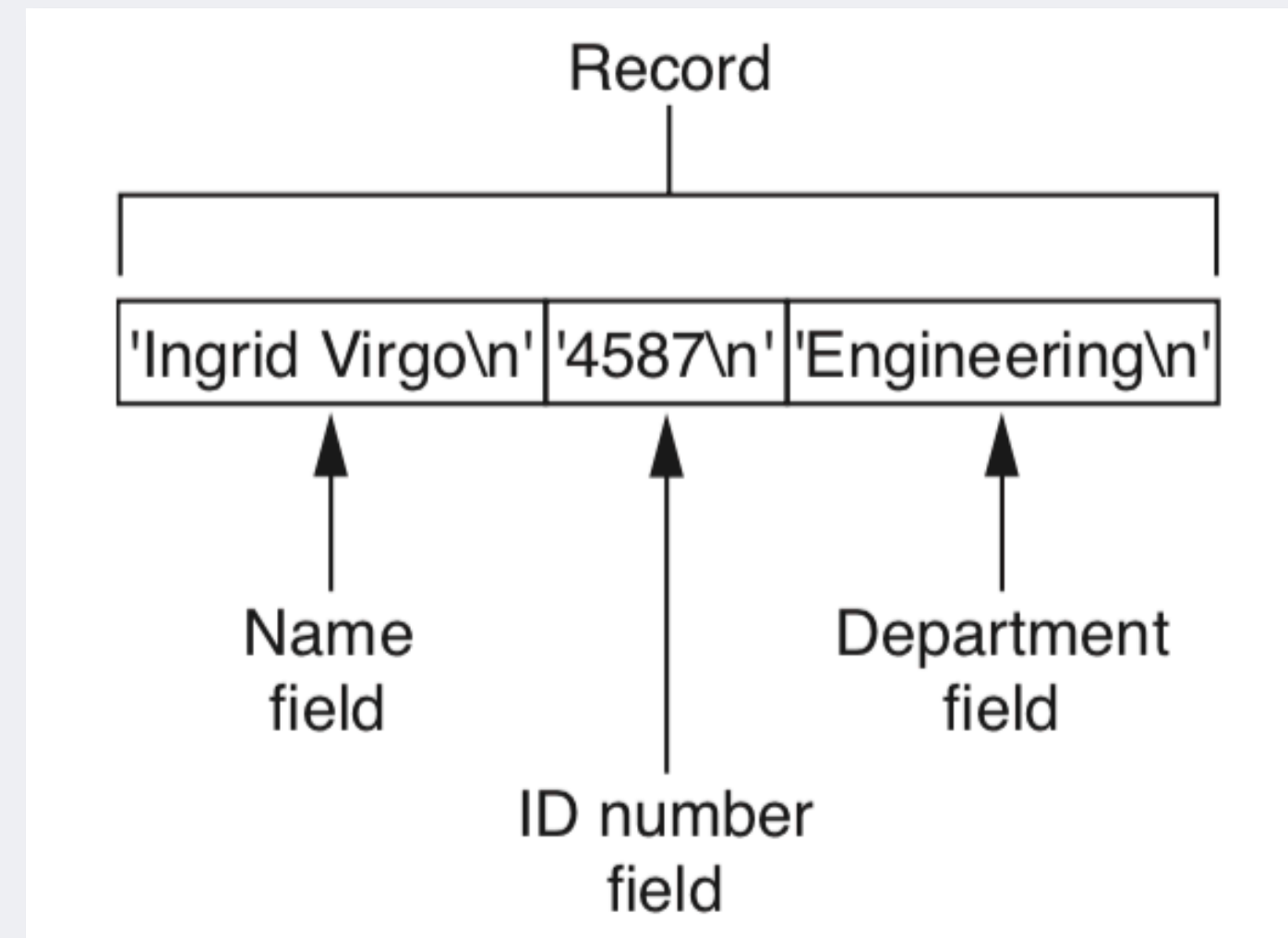


Processing Record

PROCESSING RECORDS

- **Record**: set of data that describes one item
- **Field**: single piece of data within a record
- **Write record** to sequential access file by writing the fields one after the other
- **Read record** from sequential access file by reading each field until record complete

RECORD



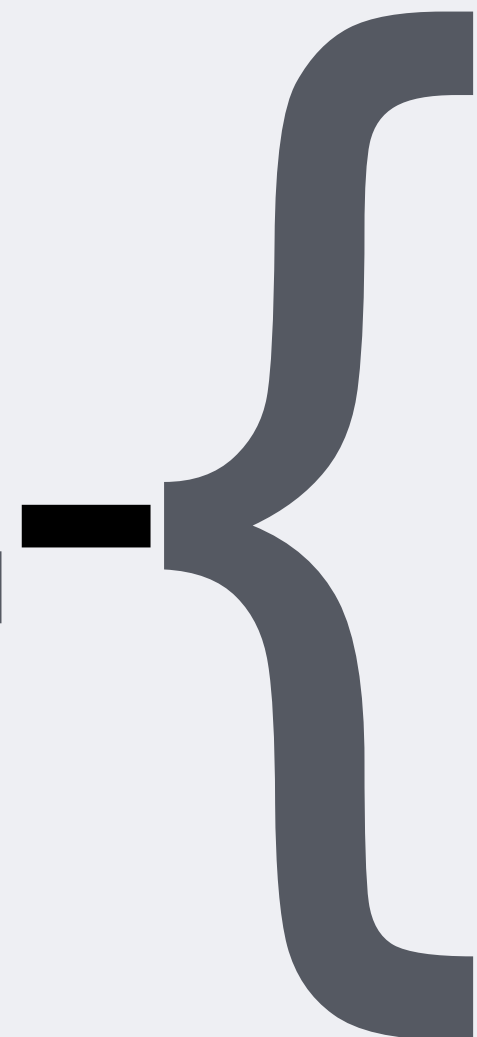
PROCESSING RECORDS

When working with records, it is also essential to be able to:

- Add records
- Display records
- Search for a specific record
- Modify records
- Delete records

SAVE_EMP_RECORDS.PY (WHILE LOOP)

Input and
write record



```
1 # Get the number of employee records to create.
2 num_emps = int(input('How many employee records do you want to create? '))
3 # Open a file for writing using with statement.
4 with open('employees.txt', 'w') as emp_file:
5     # Get each employee's data and write it to the file.
6     for count in range(1, num_emps + 1):
7         # Get the data for an employee.
8         print('Enter data for employee #', count, sep='')
9         name = input('Name: ')
10        id_num = input('ID number: ')
11        dept = input('Department: ')
12        # Write the data as a record to the file.
13        emp_file.write(name + '\n')
14        emp_file.write(id_num + '\n')
15        emp_file.write(dept + '\n')
16        # Display a blank line.
17        print()
18
19 print('Employee records written to employees.txt.')
```

Listing 8.12: Working with Text Record

SAVE_EMP_RECORDS.PY (WHILE LOOP)

Program Output (with input shown in bold)

How many employee records do you want to create? **3**

Enter the data for employee #1

Name: **Ingrid Virgo**

ID number: **4587**

Department: **Engineering**

Enter the data for employee #2

Name: **Julia Rich**

ID number: **4588**

Department: **Research**

Enter the data for employee #3

Name: **Greg Young**

ID number: **4589**

Department: **Marketing**

Employee records written to employees.txt.

EXERCISE 1: NOW! WRITE PROGRAM TO SHOW THIS

Program Output

Name: Ingrid Virgo

ID: 4587

Dept: Engineering

Name: Julia Rich

ID: 4588

Dept: Research

Name: Greg Young

ID: 4589

Dept: Marketing

IMPORT OS

```
1  import os  # Needed for the remove and rename functions
2
3
4  os.remove('employee.txt')
5
6  os.rename('temp.txt', 'coffee.txt')
```


FILES



Binary File and Record

RECORD AND BINARY FILE

การทำงานกับเรคอร์ดและไฟล์ไบนารีในภาษา Python ช่วยให้สามารถจัดการข้อมูลได้อย่างมีประสิทธิภาพยิ่งขึ้น โดยเฉพาะเมื่อเกี่ยวข้องกับข้อมูลที่มีโครงสร้างหรือไม่ใช่ข้อความ เรคอร์ดมักจัดเก็บเป็นชุดของฟิลด์ และไฟล์ไบนารีสามารถจัดเก็บเรคอร์ดเหล่านี้ในรูปแบบที่กระชับและมีประสิทธิภาพ ไฟล์ไบนารีจะแตกต่างจากไฟล์ข้อความตรงที่เก็บข้อมูลในรูปแบบไบนารีดิบ ซึ่งมีความเร็วและประหยัดพื้นที่มากกว่าเมื่อใช้กับข้อมูลบางประเภท

RECORD AND BINARY FILE

ในการเขียนเรคคอร์ดลงในไฟล์ไบนารี ต้องแปลงข้อมูลให้อยู่ในรูปแบบไบนารีก่อน โดยโมดูล `struct` ในภาษา Python มักถูกใช้เพื่อวัตถุประสงค์นี้ ซึ่งให้ฟังก์ชันในการ pack ข้อมูลเป็นไบนารี และ unpack กลับสู่รูปแบบเดิมได้

```
1 import struct
2
3 # Define a record as a tuple
4 record = (1, 'John Doe', 20, 3.75)
5 # Open a binary file for writing
6 with open("records.bin", "wb") as file:
7     # Pack the record into binary format
8     data = struct.pack('i20sif', record[0], record[1].encode(),
9                        record[2], record[3])
10    # Write the packed data to the file
11    file.write(data)
```

Listing 8.13: Writing a Record to a Binary File

WRITING RECORDS TO A BINARY FILE

คำอธิบายรูปแบบ "i20sif"

สตริงรูปแบบ "i20sif" ที่ใช้ในฟังก์ชัน `struct.pack()` ระบุโครงสร้างของข้อมูล:

- **i**: แทนจำนวนเต็ม 4 ไบต์ (32 บิต) ใช้สำหรับฟิลด์ ID และ Age
- **20s**: แทนสตริงขนาด 20 ไบต์ อักษร **s** หมายถึงสตริง และตัวเลข 20 บ่งชี้ว่าจะใช้พื้นที่ 20 ไบต์ในไฟล์ไบนารี ถ้าสตริงสั้นกว่าจะถูกเติมด้วย null byte (`\x00`) ถ้ายาวกว่าจะถูกตัด ใช้กับฟิลด์ Name
- **f**: แทนจำนวนทศนิยมขนาด 4 ไบต์ (32 บิต) ใช้กับฟิลด์ GPA

ดังนั้น "i20sif" จึงหมายถึงโครงสร้างข้อมูลประกอบด้วยจำนวนเต็ม 4 ไบต์, สตริง 20 ไบต์, จำนวนเต็ม 4 ไบต์ และทศนิยม 4 ไบต์ ซึ่งช่วยให้ข้อมูลถูกจัดเก็บอย่างเป็นระเบียบในไฟล์ไบนารี

WRITING MULTIPLE RECORDS TO A BINARY FILE

```
1 import struct
2
3 # Get the number of records to write
4 num_records = int(input("How many records do you want to create? "))
5 # Open a binary file for writing
6 with open("records.bin", "wb") as file:
7     # Loop to get each record's data
8     for _ in range(num_records):
9         # Get data from the user
10        id_num = int(input("Enter ID: "))
11        name = input("Enter Name: ")
12        age = int(input("Enter Age: "))
13        gpa = float(input("Enter GPA: "))
14        # Pack the record into binary format
15        data = struct.pack('i20sif', id_num, name.encode(), age, gpa)
16        # Write the packed data to the file
17        file.write(data)
18
19 print(f"{num_records} records have been written to records.bin")
```

Listing 8.14: Writing Multiple Records to a Binary File

READING RECORDS FROM A BINARY FILE

```
1 import struct
2
3 # Open the binary file for reading
4 with open("records.bin", "rb") as file:
5     # Read the binary data from the file
6     data = file.read(struct.calcsize('i20sif'))
7     # Unpack the data back into a tuple
8     record = struct.unpack('i20sif', data)
9     # Decode the string field and remove trailing null bytes
10    record = (record[0], record[1].decode().strip('\x00'), record[2], record[3])
11    print(f"ID: {record[0]}, Name: {record[1]}, Age: {record[2]}, GPA: {record[3]}")
    )
```

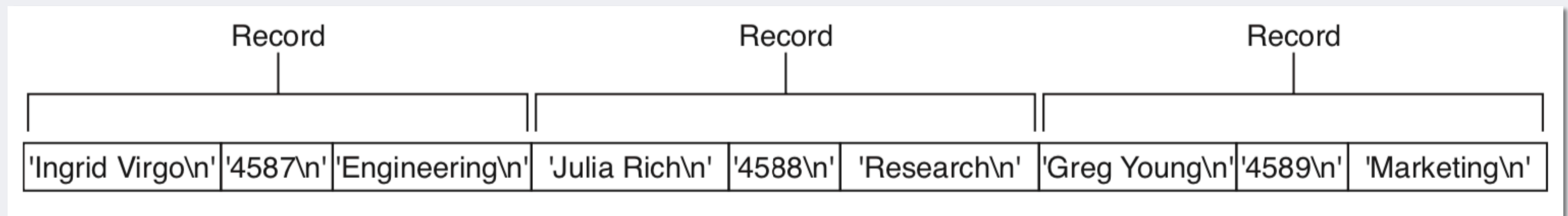
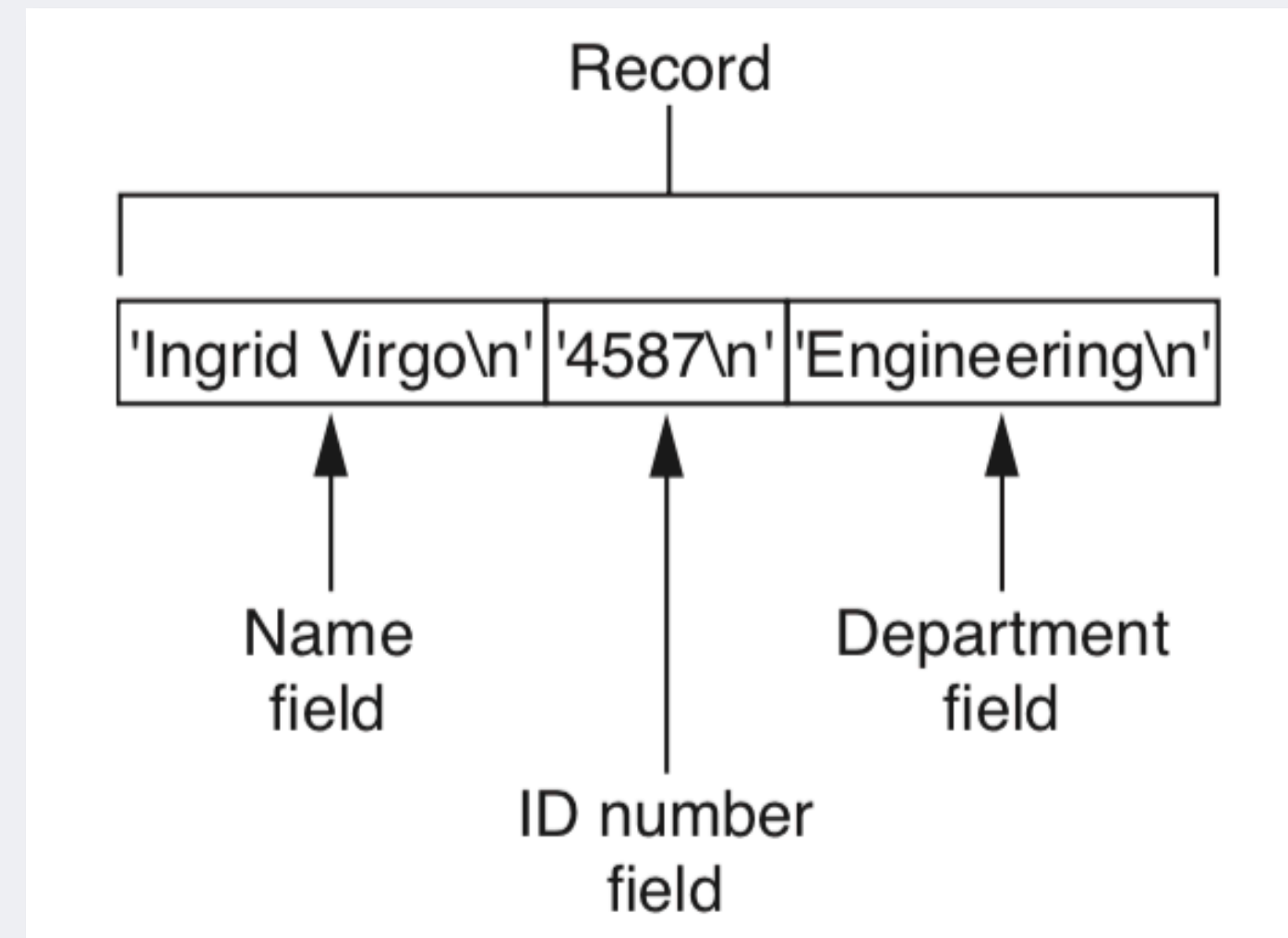
Listing 8.14: Reading a Record from a Binary File

WORKING WITH MULTIPLE RECORDS

```
1 import struct
2
3 # Open the binary file for reading
4 with open("records.bin", "rb") as file:
5     record_size = struct.calcsize('i20sif')
6     while True:
7         data = file.read(record_size)
8         if not data:
9             break
10        record = struct.unpack('i20sif', data)
11        record = (record[0], record[1].decode().strip('\x00'), record[2], record
12                  [3])
13        print(f"ID: {record[0]}, Name: {record[1]}, Age: {record[2]}, GPA: {record
14              [3]}")
```

Listing 8.15: Reading Multiple Records

RECORD



READ THE SECOND RECORD

```
1 import struct
2
3 # Define the format string
4 record_format = 'i20sif'
5 record_size = struct.calcsize(record_format)
6 # Open the binary file for reading
7 with open("records.bin", "rb") as file:
8     # Skip the first record
9     file.seek(record_size)
10    # Read the second record
11    data = file.read(record_size)
12    # Unpack the data back into a tuple
13    record = struct.unpack(record_format, data)
14    # Decode the string field and remove trailing null bytes
15    record = (record[0], record[1].decode().strip('\x00'), record[2], record[3])
16
17    print(f"ID: {record[0]}, Name: {record[1]}, Age: {record[2]}, GPA: {record[3]}")
18    )
```

Listing 8.17: Reading the second record from a binary file

BINARY FILE I/O CONSIDERATIONS

- **Endianness (ลำดับไบนารี):** ควรระวังลำดับไบนารีของระบบ (endianness) เมื่อทำงานกับไฟล์ไบนารี โดยเฉพาะกรณีที่ไฟล์จะถูกอ่านในแพลตฟอร์มที่แตกต่างกัน
- **ขนาดไฟล์:** ไฟล์ไบนารีมักมีขนาดกะทัดรัดกว่าไฟล์ข้อความ ซึ่งช่วยประหยัดพื้นที่ดิสก์และเพิ่มประสิทธิภาพการทำงาน
- **ความถูกต้องของข้อมูล:** ต้องแน่ใจว่าชนิดข้อมูลและโครงสร้างที่ใช้ในการ pack และ unpack ตรงกันอย่างถูกต้อง มิฉะนั้นข้อมูลอาจเสียหายหรือไม่สามารถอ่านได้

การใช้โหมด + ใน PYTHON

โหมดไฟล์ + ในภาษา Python ช่วยให้สามารถเปิดไฟล์เพื่อทั้งการอ่านและเขียนได้ โดยพฤติกรรมจะขึ้นอยู่กับโหมดที่ใช้งานร่วมกัน (r+, w+, หรือ a+) ดังนี้:

- **r+:** เปิดไฟล์เพื่ออ่านและเขียน ไฟล์จะต้องมีอยู่แล้ว มิฉะนั้นจะเกิดข้อผิดพลาด
- **w+:** เปิดไฟล์เพื่ออ่านและเขียน หากไฟล์มีอยู่แล้วจะลบเนื้อหาเก่าออก หากยังไม่มีจะสร้างไฟล์ใหม่
- **a+:** เปิดไฟล์เพื่ออ่านและเขียน ตัวชี้ตำแหน่งไฟล์จะอยู่ที่ท้ายไฟล์ (หากมีอยู่) การเขียนจะถูกเพิ่มต่อท้ายเสมอ หากไฟล์ยังไม่มีจะสร้างไฟล์ใหม่

การใช้โหมด + ใน PYTHON

ตัวอย่างต่อไปนี้แสดงการใช้โหมด w+ เพื่อสร้างไฟล์ใหม่ เขียน และอ่านจากไฟล์:

```
1 # Example using w+ mode (write and read)
2 def example_w_plus_mode():
3     # Open the file for reading and writing (truncating it)
4     with open('example_w+.txt', 'w+') as file:
5         # Write some content to the file
6         file.write("This is the first line in the file.\n")
7         file.write("This is the second line in the file.\n")
8
9         # Move the file pointer back to the beginning
10        file.seek(0)
11
12        # Read the content of the file
13        content = file.read()
14        print("Content of the file:")
15        print(content)
16
17 example_w_plus_mode()
```

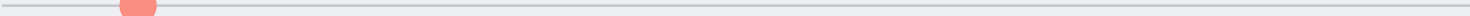
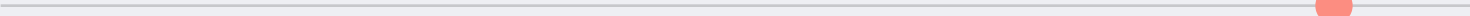
Listing 8.19: Example using w+ mode (write and read)

การใช้โหมด + ใน PYTHON

ตัวอย่างต่อไปนี้แสดงการใช้โหมด a+ เพื่อเปิดไฟล์ เพิ่มข้อมูล และอ่านเนื้อหาจากไฟล์:

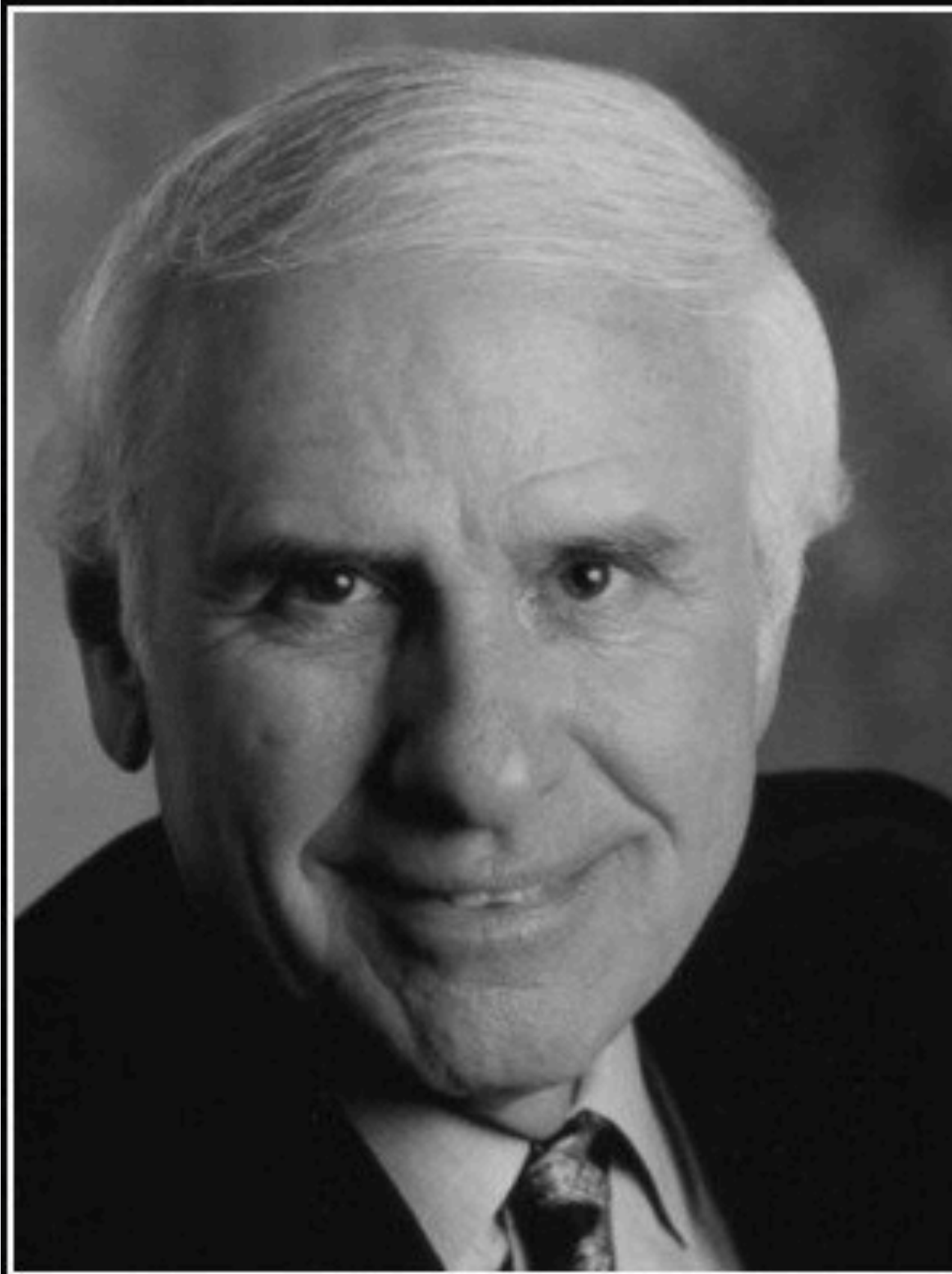
```
1 # Example using a+ mode (append and read)
2 def example_a_plus_mode():
3     # Open the file for reading and writing (appending)
4     with open('example_a+.txt', 'a+') as file:
5         # Move the file pointer to the beginning to read the content
6         file.seek(0)
7
8         # Read the current content of the file
9         content = file.read()
10        print("Current content of the file:")
11        print(content)
12
13        # Append new content to the file
14        file.write("Appending a new line at the end.\n")
15
16        # Move the file pointer to the beginning again to read the updated
17        # content
18        file.seek(0)
19        updated_content = file.read()
20        print("\nUpdated content of the file:")
21        print(updated_content)
22
23 example_a_plus_mode()
```

Listing 8.20: Example using a+ mode (append and read)



THANKS

WORD OF THE WISE



Don't join an easy crowd; you won't grow. Go where the expectations and the demands to perform are high.

— *Jim Rohn* —

AZ QUOTES